Ansible — Roles, Variables & AWS EC2

Assignment | Q1: Roles Reusability & Variables Q2: Configure & Launch EC2 on AWS

Q1 Roles — Reusability & Variables

Organising automation for scale and reuse

What are Roles?

In Ansible, a **Role** is a structured, self-contained unit of automation. Rather than writing one long playbook that does everything, you break your automation into roles — each role handles one concern (e.g. installing a web server, setting up a database, or hardening a server). A role has a standard folder structure that Ansible recognises automatically:

tasks/	The main list of tasks the role executes.
handlers/	Tasks triggered by 'notify' (e.g. restart service on config change).
vars/	Variables specific to this role (higher precedence).
defaults/	Default variable values — easiest to override from outside.
templates/	Jinja2 template files (.j2) used by the template module.
files/	Static files copied to managed hosts.
meta/	Role metadata — author, dependencies on other roles.

Reusability

The biggest advantage of roles is **reusability**. Once you write a role for, say, installing and configuring Nginx, you can reuse it across dozens of playbooks and projects without copying a single line of code. Teams share roles via **Ansible Galaxy** (the community hub), or host them in private repositories. A playbook that uses roles becomes clean and declarative — it simply lists *which* roles to apply to which hosts, not *how* to do it.

Variables

Variables make roles flexible. Instead of hard-coding values like port numbers, usernames, or file paths, you define them as variables and pass in the right value for each environment. Ansible resolves variables in a specific **precedence order** — from lowest to highest:

role defaults ■	inventory vars ■	playbook vars ■	**extra vars (-e) ★**

★ ***Extra vars*** *(passed at runtime with -e) always win — perfect for CI/CD pipelines where you need to override a value without touching the role itself.*

Q2 Configure & Launch EC2 on AWS

Using Ansible to provision cloud instances automatically

1 Install Prerequisites

Install Ansible on your control machine and add the AWS collection: **ansible-galaxy collection install amazon.aws**. Also install the **boto3** and **botocore** Python libraries (*pip install boto3 botocore*) — Ansible uses these to talk to AWS APIs.

2 Configure AWS Credentials

Set your AWS access keys so Ansible can authenticate. The cleanest way is via environment variables (**AWS_ACCESS_KEY_ID** and **AWS_SECRET_ACCESS_KEY**) or the *~/.aws/credentials* file. For production, use an IAM Role attached to your control machine instead of long-lived keys.

3 Write the EC2 Playbook

Create a YAML playbook that uses the **amazon.aws.ec2_instance** module. Specify the AMI ID, instance type (e.g. *t2.micro*), key pair, security group, subnet, region, and any tags. You can define all these values as variables so the same playbook works across environments (dev, staging, production).

4 Define Security Group & Key Pair

Use **amazon.aws.ec2_security_group** to create or reference a security group that allows SSH (port 22) and any app ports you need. Ensure your EC2 key pair exists in AWS — reference it by name in the playbook. Ansible can also create the key pair and save the private key locally using the **amazon.aws.ec2_key** module.

5 Launch & Wait for the Instance

Run the playbook with **ansible-playbook launch_ec2.yml**. Ansible calls the AWS API to create the instance and returns the public IP. Use the **wait: true** option so Ansible pauses until the instance reaches the 'running' state before moving to the next task.

6 Add Instance to Inventory & Provision

Register the new instance's public IP as a host using **add_host**, then continue in the same playbook (or a follow-up playbook) to SSH in and configure the server — install packages, deploy your app, apply roles. This turns a raw EC2 instance into a fully configured, ready-to-use server in a single automated run.

Roles	Variables	EC2 Steps
Reusable task units	*Dynamic, overridable config*	*Install → Creds → Playbook → Launch → Provision*